



C++ Programming Bootcamp 2

COSC2802

Topic 12 Inheritance

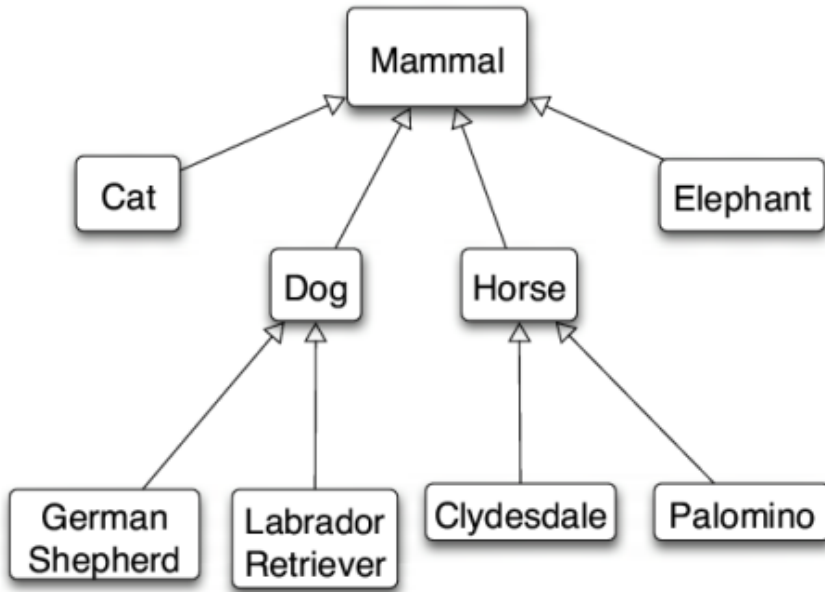




Derived Classes



Example



Derived Class Declaration

Example:

- Derived class, *Dog*, inherits from *Mammal*. Declare as:

- Derived class name
- Followed by colon (:)
- Followed by access specifier **
- Followed by base class name

Visibility of base class members:
determines how members of base
class (*Mammal*) are inherited

```
class Dog : public Mammal
{
public:
    void print()
    {
        Mammal::print();
        std::cout << "I am a dog" << std::endl;
    }
};
```

Controls access to members
within this class (*Dog*)

** access specifier e.g. public (more on this later)

```

public:
    void print()
    {
        std::cout << "I am a mammal" << std::endl;
    }
};

class Cat : public Mammal
{
public:
    void print()
    {
        Mammal::print();
        std::cout << "I am a cat" << std::endl;
    }
};

class Dog : public Mammal // if "public" is omitted - implicitly declared private
// class Dog : Mammal
{
public:
    void print()
    {
        Mammal::print();
        std::cout << "I am a dog" << std::endl;
    }
};

class Labrador : public Dog
{
public:
    void print()
    {
        Dog::print();
        std::cout << "I am a labrador" << std::endl;
    }
};

```

```

Mammal m1;
Cat c1;
Dog d1;
Labrador l1;

std::cout << "\nMammal:\n";
m1.print();

std::cout << "\nCat:\n";
c1.print();

std::cout << "\nDog:\n";
d1.print();

std::cout << "\nLabrador:\n";
l1.print();

```

```

Mammal:
I am a mammal

Cat:
I am a mammal
I am a cat

Dog:
I am a mammal
I am a dog

Labrador:
I am a mammal
I am a dog
I am a labrador

```

Different Types of Access Modifiers (for classes)

Used **within Class** to control visibility/access of members (data and functions)

Different types of access:

- Private - Accessible only within the class NOT to members of derived classes
- Protected - Accessible from within the class AND to derived classes
- Public - Accessible to class AND derived classes AND anywhere else

NOTE Difference between:

- Keyword **in class**
 - Specifies access level of the method(s) or function(s)
 - We will consider all levels

```
class Dog : public Mammal
{
    public:
        void print()
        {
            Mammal::print();
            std::cout << "I am a dog" << std::endl;
        }
};
```

Different Types of Access Specifiers (for Inheritance)

Different types of access:

- **Public Inheritance** – Retains visibility of base class members in derived class i.e. public/private/protected members remain public/private/ in derived class
- **Protected** – Use to limit access to inherited members ie public/protected become protected in derived class; private remain private in derived class
- **Private** – Used to restrict so inherited members aren't accessible outside derived class i.e. all members become private

NOTE Difference between:

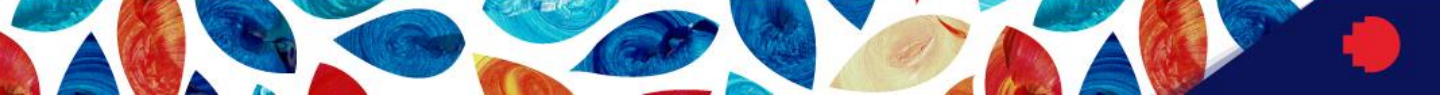
- Keyword in class
 - Specifies access level of the method(s)
 - We will consider all levels
- Keyword **before base class**:
 - Specifies *type of inheritance*: access level of inherited members base class
 - **We will only consider public**

```
class Dog : public Mammal
{
public:
    void print()
    {
        Mammal::print();
        std::cout << "I am a dog" << std::endl;
    }
};
```



Polymorphism





Polymorphism

Polymorphism: Determining which program behavior to execute depending on data types. Two main types of polymorphism exist:

1. **Compile-time polymorphism (aka static binding):** compiler determines which function to call at compile-time. Example:
 - Function **overloading**: compiler determines which of several identically-named functions to call based on the function's arguments (different parameter types, number of parameters, or return values).
2. **Runtime polymorphism (dynamic binding):** compiler unable to determine which function to call at compile-time and can only be determined at runtime.
 - Function **Overriding** with:
 - **virtual functions**
 - **base class pointers**

1. Compile-time Polymorphism

Compile-time polymorphism (aka static binding)

- the compiler determines which function to call at compile-time e.g. function **overloading**
- Resolve based on:
 - Function name, parameter type(s), number of parameters, default arguments,

```
void DriveTo(string restaurant) {  
    cout << "Driving to " << restaurant << endl;  
}  
  
void DriveTo(Restaurant restaurant) {  
    cout << "Driving to " << restaurant.GetDescription() << endl;  
}  
  
int main() {  
    DriveTo("Big Mac's"); // Call string version  
}
```

2. Runtime Polymorphism

Compiler can't determine which function to call – decision made while program running
e.g. vector of **pointers** to objects of both base (business) and derived (restaurant) classes

```
void DriveTo(Business* businessPtr) {  
    cout << "Driving to " << businessPtr->GetDescription() << endl;  
}  
  
int main() {  
    int index;  
    vector<Business*> businessList; ← Vector of ptr to base class  
    Business* businessPtr; ← base class object  
    Restaurant* restaurantPtr; ← derived class object  
    ...  
    businessList.push_back(businessPtr);  
    businessList.push_back(restaurantPtr); ← Implicit conversion  
  
    index = rand() % businessList.size();  
    DriveTo(businessList.at(index));  
}
```

?? Which function ??

Overriding Member Functions (zybooks)

Overriding, a derived class member function with same name must have:

- same parameter types, number parameters, and return value as base class member function

Friends Cafe -- 500 W 2nd Ave
Rating: 5

Note difference to:

Overloading: functions with the same name BUT:

- different parameter types
- number of parameters
- or return values.

```
class Business {
public:
    ...
    string GetDescription() const { ←
        return name + " -- " + address;
    }

protected:
    string name;
    string address;
};

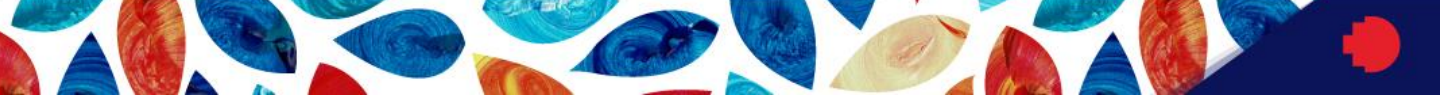
class Restaurant : public Business {
public:
    ...
    string GetDescription() const { ←
        return name + " -- " + address +
            "\n Rating: " + to_string(rating);
    }

private:
    int rating;
};

int main() {
    Restaurant favoritePlace;
    favoritePlace.SetName("Friends Cafe");
    favoritePlace.SetAddress("500 W 2nd Ave");
    favoritePlace.SetRating(5);
    cout << favoritePlace.GetDescription() << endl;

    return 0;
}
```

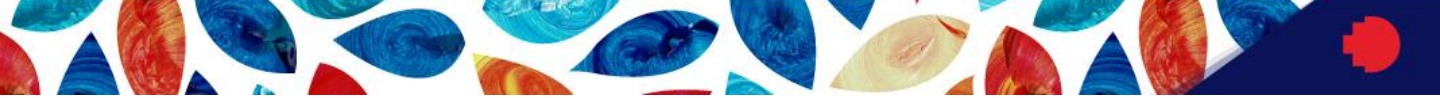
same



Objects of Base and Derived Classes?

One of the key points of inheritance is the ability to treat derived classes similarly or exactly the same as the base class. But if a derived class overrides a member function of the base class, how does the compiler know which version of a function to call?

Polymorphism!

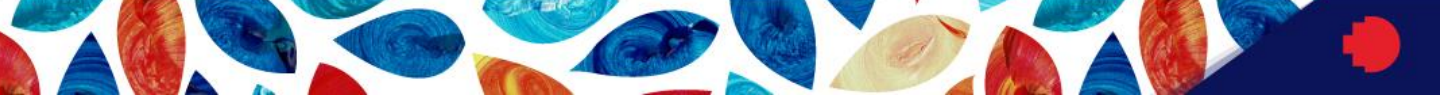


Objects of Base and Derived Classes?

One of the key points of inheritance is the ability to treat derived classes similarly or exactly the same as the base class. But if a derived class overrides a member function of the base class, how does the compiler know which version of a function to call?

Polymorphism!

- **Derived** classes: provide specific implementation of **virtual function** declared in **Base Class**
 - Default behaviour in Base Class
 - Derived classes customize behaviour for more specific cases
- **Dynamic selection** of implementation of function **at runtime**
- **Consistent Interface:** derived classes conform to interface of base
 - ie same name, return type, parameters = consistent behaviour



Virtual Functions

A virtual function is a function that we expect all derived classes to override.

The purpose of the `virtual` keyword is to tell the compiler that derived classes may override this function, and that the derived version should be called instead of the base class's function

Additionally: The **`override`** keyword can be used in derived classes to make it clear that the function overrides the base class's implementation of the function.

Virtual Example



```
class Animal
{
    public:
        void eat() { std::cout << "I'm eating generic food." << std::endl; }
};

class Cat : public Animal
{
    public:
        void eat() { std::cout << "I'm eating cat food." << std::endl; }
};
```

```
void func(Animal *a) { a->eat(); }
```

```
Animal *animal = new Animal;
Cat *cat = new Cat;

animal->eat(); // Outputs: ??
cat->eat();    // Outputs: ??

func(animal); // Outputs: ??
func(cat);    // Outputs: ??
```

Virtual Example



```
class Animal
{
    public:
        void eat() { std::cout << "I'm eating generic food." << std::endl; }
};

class Cat : public Animal
{
    public:
        void eat()          { std::cout << "I'm eating cat food." << std::endl; }
};
```

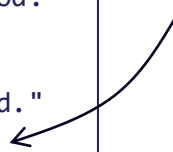
```
void func(Animal *a) { a->eat(); }
```

```
Animal *animal = new Animal;
Cat *cat = new Cat;

animal->eat(); // Outputs: "I'm eating generic food."
cat->eat();    // Outputs: "I'm eating cat food."

func(animal); // Outputs: "I'm eating generic food."
func(cat);    // Outputs: "I'm eating generic food."
```

Want it to use the eat()
function from Cat



Virtual Example



```
class Animal
{
    public:
        virtual void eat() { std::cout << "I'm eating generic food." << std::endl; }
};

class Cat : public Animal
{
    public:
        void eat() override { std::cout << "I'm eating cat food." << std::endl; }
};
```



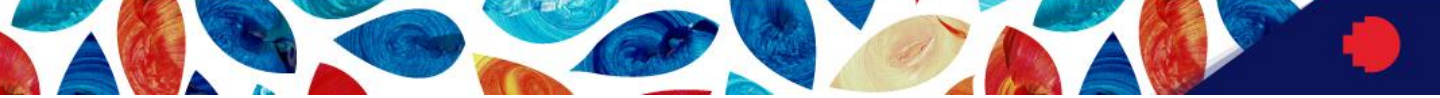
- Ensures function meant to override
- Readability (self-documenting code)
- Compile time check (matching virtual function in base class?)

```
void func(Animal *a) { a->eat(); }
```

```
Animal *animal = new Animal;
Cat *cat = new Cat;

animal->eat(); // Outputs: "I'm eating generic food."
cat->eat();    // Outputs: "I'm eating cat food."

func(animal); // Outputs: "I'm eating generic food."
func(cat);    // Outputs: "I'm eating cat food."
```



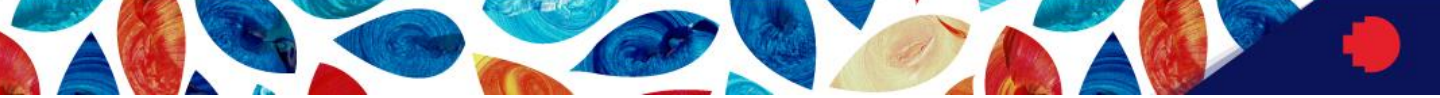
Runtime Polymorphism

Allows functions to be called based on

- **dynamic type** of the object at runtime, rather than the **static type** at compile time.

Enabled through:

- **Virtual Functions:**
 - Declared in the base class using the **virtual** keyword, overridden in derived classes.
- **Base Class Pointers or References for Dynamic Type Resolution:**
 - Pointers (or references) are used because they allow the program to decide at runtime which function implementation to use, based on the type of the object being pointed to
 - This is determined by the dynamic type of the object, using the **vtable** (virtual table) and **vpitr** (virtual pointer) to ensure correct overridden function is called based on the dynamic type of the object



Object Slicing

Assignment by Value (Object Slicing) When a derived class object is assigned to a base class by value (ie **not** using pointers or references, object slicing occurs e.g.

```
Cat cat;  
Animal animal = cat; // animal only retains the base class part  
animal.eat();
```

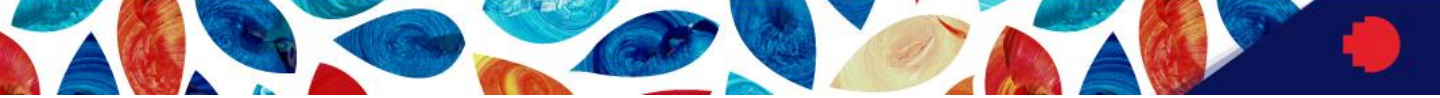
the derived part of the object (attributes and methods specific to the derived class) is "**sliced off**," leaving only the base class portion (will use the animal function)

Assignment by Reference or Pointer (Preserves Dynamic Behaviour) Using a pointer or reference to the base class avoids object slicing and allows full access to derived-specific behaviour e.g. will use the cat function (the type of object the pointer points to)

```
Cat cat;  
Animal* animal = &cat; // animal points to Cat object retains dynamic type info  
animal.eat();
```

Example Use:

Collection of **base and derived objects** – select the appropriate function without object slicing



Example

```
class Base {  
public:  
    virtual void display() { cout << "Base display" << endl; }  
};  
class Derived : public Base {  
public:  
    void display() override { cout << "Derived display" << endl; }  
    int derivedVar; // Additional member in Derived  
};
```

Example (avoid object slicing)

```
class Base {
public:
    virtual void display() { cout << "Base display" << endl; }
};
class Derived : public Base {
public:
    void display() override { cout << "Derived display" << endl; }
    int derivedVar; // Additional member in Derived
};

int main() {
    Derived d;
    d.derivedVar = 42;

    Base b = d; // Object slicing occurs here

    b.display(); // Calls Base::display(), Derived::display() is inaccessible

    return EXIT_SUCCESS;
}
```

Example

```
class Base {
public:
    virtual void display() { cout << "Base display" << endl; }
};
class Derived : public Base {
public:
    void display() override { cout << "Derived display" << endl; }
    int derivedVar; // Additional member in Derived
};

int main() {
    Derived d;
    d.derivedVar = 42;

    Base b = d; // Object slicing occurs here
    Base* basePtr = &d; // Preserve dynamic type (allow access to overridden function)
    b.display(); // Calls Base::display(), Derived::display() is inaccessible
    b->display(); // Polymorphic call, resolves to Derived::display()
    return EXIT_SUCCESS;
}
```

Example: collection of base and derived objects



```
class Business {
public:
    virtual string GetDescription() const {

    }
    ...
};

class Restaurant : public Business {
public:
    string GetDescription() const override {
        return name + " -- " + address +
            "\n Rating: " + to_string(rating);
    }
    ...
};
```

Use:

- Collection of base and derived objects

e.g. w/s exercise “Animal Inherited vectors”

```
void DriveTo(Business* businessPtr) {
    cout << "Driving to " << businessPtr->GetDescription() << endl;
}

int main() {
    int index;
    vector<Business*> businessList;
    Business* businessPtr;
    Restaurant* restaurantPtr;
    ...
    businessList.push_back(businessPtr);
    businessList.push_back(restaurantPtr);

    index = rand() % businessList.size();
    DriveTo(businessList.at(index));
}
```



‘has-a’ vs ‘is-a’



Composition/aggregation ('has-a' relationship)

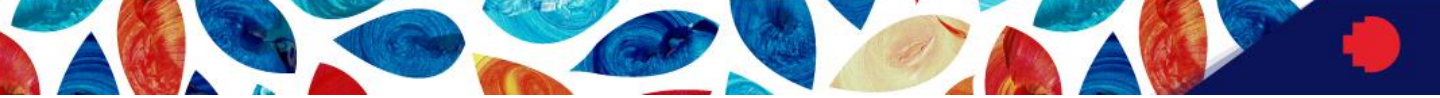
Software reuse technique where:

- a class can have **objects of other classes** as members

Example: car class with multiple parts such as engine, wheels, ...

- Each part modelled as **separate class**
- Car class has composition (has-a) relationship with these parts

```
class Engine {  
|   // implementation goes here  
};  
  
class Wheel {  
|   // implementation goes here  
};  
  
class Car {  
private:  
    Engine engine;  
    std::array<Wheel, 4> wheels;  
    // other parts go here  
  
public:  
    // Car methods and functionality  
};
```



Inheritance/derived classes ('is-a' relationship)

Software reuse technique: class is similar to another class with some additions/variations

Derived class (or **subclass**) is derived from a **base** class (or **superclass**):

- Derived class inherits properties of the base class = inheritance
- Derived class objects are also objects of the base class
- Inheritance relationships form class hierarchies

Example class:

- Base class:
 - **Vehicle**
- Derived Class(es):
 - **Car**
 - **Jeep**
 - **Bus**
 - ...

Already familiar with this from Java/Programming Bootcamp 1

